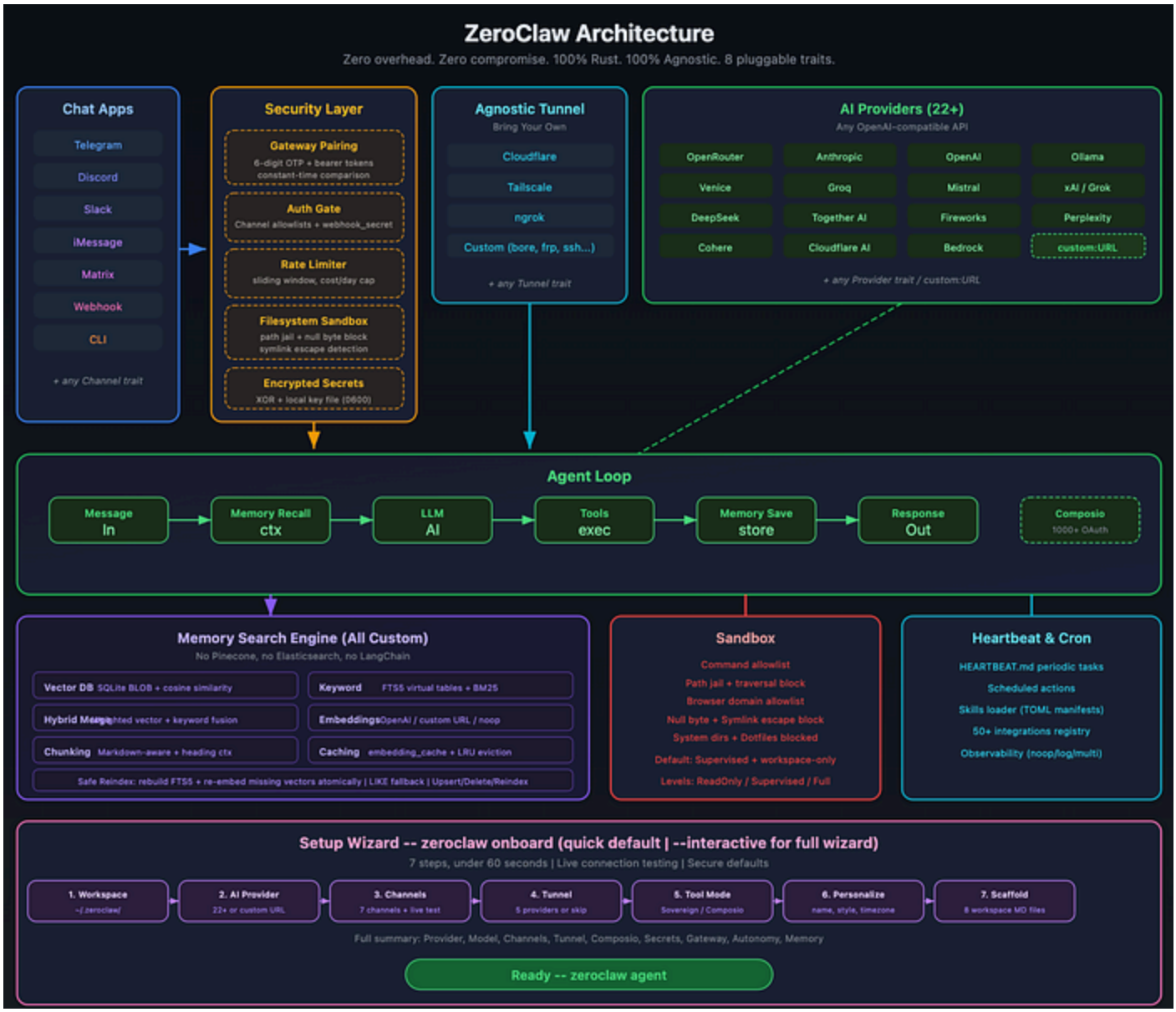
Milestone: 20GB Reached

We've reached 20GB of stored data — thank you for helping us grow!

Patreon Ko-fi Liberapay
Close
Freedium

- ko-fi
- librepay
- patreon

< Go to the original



OpenClaw Variants on $10 Hardware and 10MB RAM

Most product bugs show up when a simple feature lands on a box with a 64MB RAM budget and a watchdog timer.

Agent Native Follow

~8 min read · February 19, 2026 (Updated: February 19, 2026) · Free: No

That's why there are **light-weight, optimized variants of Openclaw** that eat **a minimum ~10MB of RAM.**

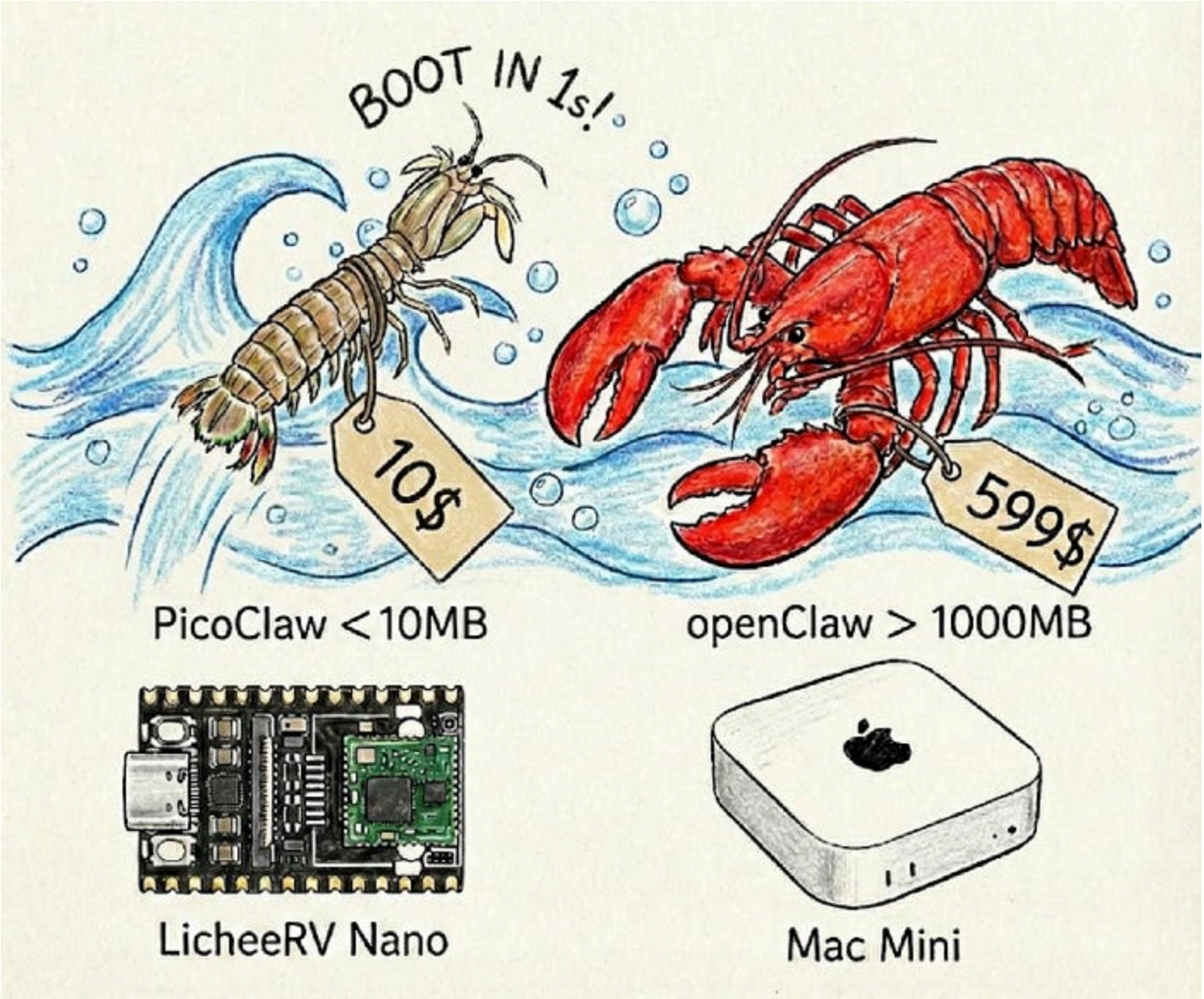
Not 10MB more than your app. Not 10MB after warmup. Just... 10MB-ish to exist.

The t**hree separate variants, Picoclaw**, **Zeroclaw**, and **Nanobot,** each seemingly taking a different swing at the same constraint.

Metric	OpenClaw	NanoBot	PicoClaw	ZeroClaw
Language	TypeScript	Python	Go	Rust
RAM	> 1GB	> 100MB	< 10MB	< 5MB
Startup (0.8GHz core)	> 500s	> 30s	< 1s	< 10ms
Binary Size	~28MB (dist)	N/A (Scripts)	~8MB	3.4 MB
Cost	Mac Mini $599	Linux SBC ~$50	Linux Board $10	Any hardware $10

The tension is **you want capability**, but **your deployment environment wants austerity**.

Let me share the exact harness I use to compare RAM footprints across builds, because in practice **10MB** can mean three very different things depending on what's counted and when.



Edge variants are more important than you think

For a long time, AI product quietly assumed:

- plentiful RAM
- GPU availability (or at least a beefy CPU)
- large runtimes and dependency trees
- containers everywhere
- observability stacks that cost more memory than your actual model

That assumption is getting stress-tested from both sides:

1. **Product pressure:** teams want AI features everywhere, on edge devices, internal tools, embedded kiosks, low-tier servers, sidecars, or customer environments you don't control.
2. **Economic pressure:** memory is a recurring cost. Not just on embedded devices but also in multi-tenant services, serverless functions, and horizontally scaled workers.
3. **Reliability pressure:** OOM kills, allocator fragmentation, slow leaks, and works-on-my-machine caches are the stuff that light up your pager.

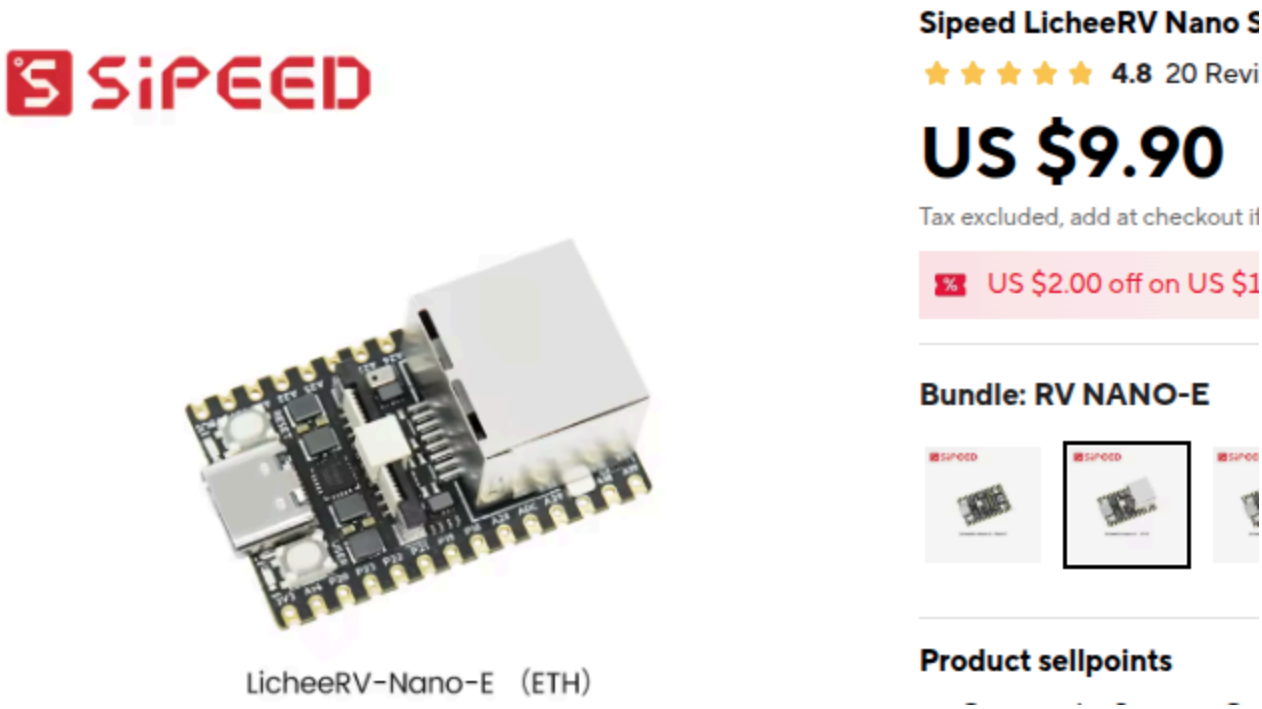
So when I see **minimum ~10MB RAM** attached to optimized variants of something (OpenClaw), my brain reads it as:

> *"Someone got tired of the default stack being too heavy to ship."*

Even if you never deploy to an actual edge device, the design discipline that gets you into 10MB territory tends to produce systems that are:

- cheaper to run
- faster to cold start
- less fragile under load spikes
- easier to reason about

Not always. But often.



> *I'm writing a deep-dive ebook on **Agentic SaaS**, the emerging design patterns that are quietly powering the most innovative startups of 2026.*
>
> ***You can grab the first chapter here:** [Agentic SaaS Patterns Winning in 2026](), packed with real-world examples, architectures, and workflows you won't find anywhere else.*

Light-weight Variants of OpenClaw

There are three **light-weight optimized variants of OpenClaw** that use **minimum ~10MB of RAM**

1. Picoclaw
2. Zeroclaw
3. Nanobot

I previously wrote about Nanobot:

Clawdbot Lite: 99% Smaller, 4000 Lines, Same Core Power

People are building Polymarket bots, multi-agent product teams, crypto traders, coding agents, business operators with...

medium.com

And if you want the repositories, here they are:

```
Picoclaw:  https://github.com/sipeed/picoclaw
Zeroclaw:  https://github.com/zeroclaw-labs/zeroclaw
Nanobot:   https://github.com/HKUDS/nanobot
```

Copy

Three variants implies optimization is not one trick

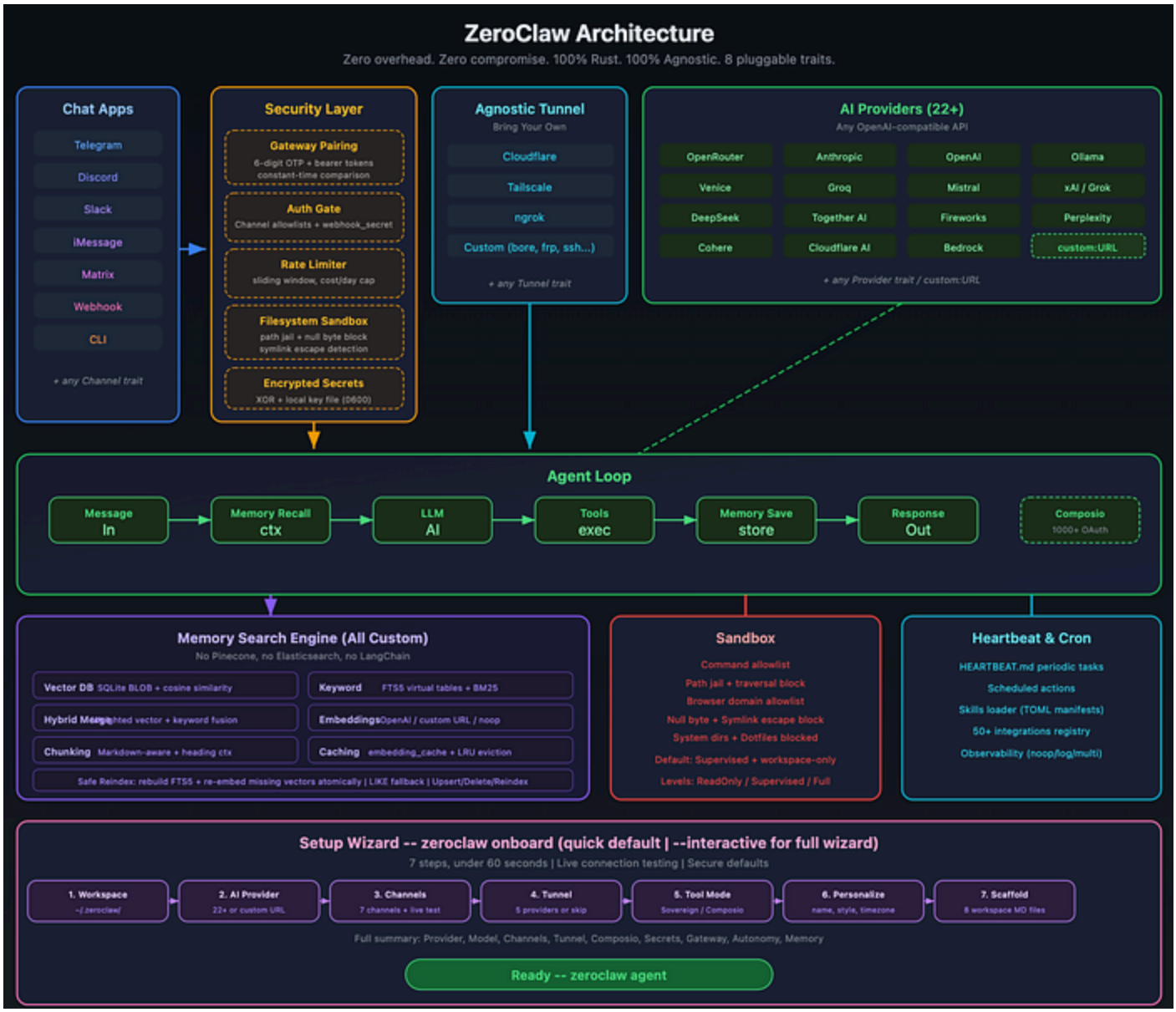
If this were a single lite repo, I'd assume it's mostly pruning and compilation flags.

But **three** separate light-weight variants suggests:

- different tradeoffs were made
- different constraints were prioritized
- maybe different target environments exist (even if we don't know which ones)

In other words: **memory optimization is a design space.**

Here's a quick look at Zeroclaw's architecture



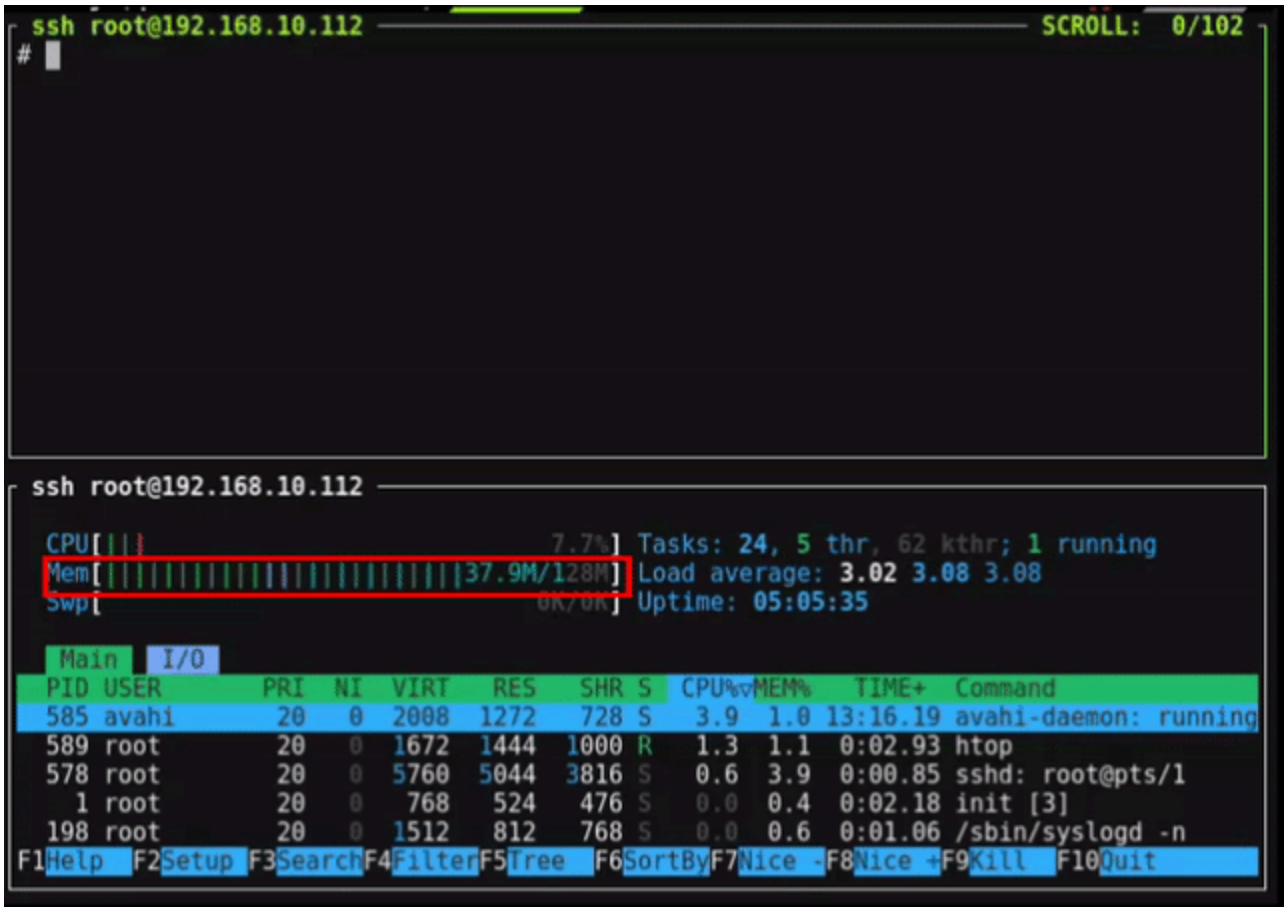
If you there are three optimized variants, it's usually because:

- upstream dependencies are too large
- default runtime behavior is too memory-hungry
- or the baseline architecture wasn't designed for constrained environments

10MB thinking or lower memory variants force you to confront the parts of AI engineering usually ignore until production burns you:

- hidden runtime overhead
- incidental memory allocations
- caching defaults
- tokenization/serialization bloat
- logging and tracing payloads
- concurrency multipliers
- and the classic: "we load it once per request"

Here are three concrete scenarios where **lightweight variants** matter immediately.



(1) You're shipping an AI sidecar in a multi-tenant environment

You run one instance per customer, per namespace, per node.

A 10–50MB footprint becomes "we can actually scale this sanely."

(2) Your cold starts are killing UX

In serverless or autoscaling, memory footprint is tightly coupled with:

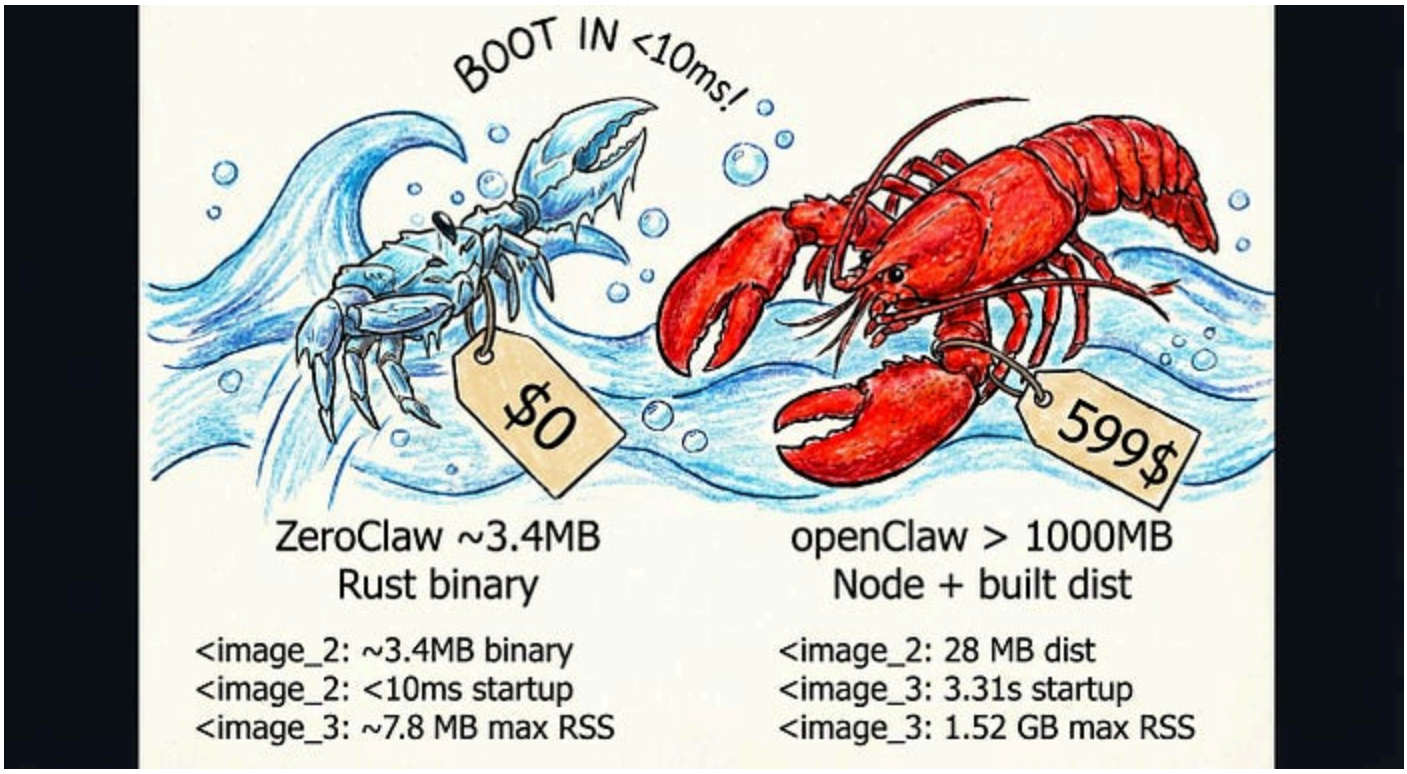
- startup time
- container image size (often correlated)
- time-to-first-token/time-to-first-response

(3) You want AI features on constrained customer hardware

Routers. Industrial PCs. Thin VMs. "Bring-your-own-infra" deployments. Air-gapped environments.

Even if you *can* ask customers to allocate more RAM, doing so is a tax:

- procurement delays
- higher price points
- more failure modes



How to evaluate

I'm choosing based on:

- **measured memory behavior**
- **failure modes under pressure**
- **operational fit**
- **maintainability risk**

Here's a minimal harness I use to compare memory footprints across binaries or scripts.

Step 1: Measure peak RSS and runtime (Linux)

```
#!/usr/bin/env bash
set -euo pipefail

CMD="${1:?Usage: bench_mem.sh '<command to run>'}"
# /usr/bin/time -v prints "Maximum resident set size (kbytes)"
/usr/bin/time -v bash -lc "$CMD" 2>&1 | tee /tmp/mem_bench.out
echo
echo "---- Extract ----"
grep -E "Maximum resident set size|Elapsed" /tmp/mem_bench.out || true
```

Usage examples (you'll replace the commands with how each variant runs in your environment):

```
./bench_mem.sh "./picoclaw --help"
./bench_mem.sh "./zeroclaw --version"
./bench_mem.sh "./nanobot --help"
```

Step 2: Enforce a memory ceiling to reveal cliffs early

This is my favorite trick because it surfaces pathological allocations fast:

```
# Limit virtual memory to ~20MB (value is KB)
ulimit -v 20480
./your_command_here
```

This isn't a perfect model of RSS (virtual memory limits are blunt), but it's a fast way to see:

- does the system fail gracefully?
- or does it crash mid-flight with unclear errors?

Step 3: Test concurrency multipliers

Even if the baseline is ~10MB, your real footprint is often:

$$baseline + (buffer_per_request \times in_flight_requests)$$

So I run something like:

```
for c in 1 2 4 8 16; do
  echo "=== concurrency=$c ==="
  ./bench_mem.sh "./your_service --concurrency $c --run-sample-workload"
done
```

If you want a lightweight AI component to survive production, here's what I check.

Memory behavior

- Peak RSS measured under representative load
- Memory stable over time (no slow leaks across minutes/hours)
- Caches have explicit size limits (no "unbounded until OOM")
- Concurrency scaling understood and tested

Operational behavior

- Fails gracefully under memory pressure (clear errors, no corruption)
- Startup cost measured (cold start matters)
- Logging/tracing can be tuned down without code changes
- No hidden background threads that allocate unpredictably

Product risk

- Active maintenance signals (issues, commits, releases)
- Clear licensing and ownership alignment
- Integration surface matches your stack (CLI, library, service)

A tiny table of tradeoffs

This is **general background knowledge** and applies to most "lite" variants:

Optimization Move	What You Gain	What You Often Give Up
Fewer dependencies	smaller footprint, fewer CVEs	less plug-and-play integration
Aggressive caching limits	predictable memory	potentially slower repeated calls
Static allocation / reuse buffers	less fragmentation, stable RSS	more complex code paths
Reduced observability by default	lower overhead	harder debugging unless configurable
Lower-level implementation	speed + memory control	higher maintenance burden

So… which of the three should you pick?

I can tell you how I'd frame the decision in a way that respects both engineering reality and product needs.

1) Decide what "Openclaw compatibility" actually means for you

Open question:

- Do you need drop-in API compatibility with baseline Openclaw?
- Or do you just need the *capability* in a smaller envelope?

2) Decide what kind of system you're building

- Embedded-ish? You care about deterministic memory and predictable failure modes.
- Multi-tenant service? You care about per-instance overhead and concurrency scaling.
- Client-side app? You care about startup cost, battery, and background memory.

3) Treat the three variants as a portfolio, not a beauty contest

The most product-minded move might be:

- prototype with two of them
- run the same harness
- measure memory + latency + failure modes
- and pick the one that fits your constraints *today*

Then keep the others on your radar as contingency options.

My slightly opinionated take

If you've never shipped into tight memory budgets, it's tempting to optimize last.

Once your architecture depends on a heavyweight runtime, your product roadmap starts inheriting infra requirements:

- larger nodes

- fewer deployment targets
- more operational fragility
- more cost to scale
- and more "it works in staging" lies

So when I see **three independent lightweight variants** clustered around **a ~10MB floor**, I see a community (or ecosystem) converging on a truth:

> *The default stack is too fat for where people want to ship next.*

Even if OpenClaw itself is something totally different than what I'm imagining, the pattern is stable: **constraints drive innovation** and the best product teams make those constraints explicit early.

Concluding thoughts

You don't need to be building for microcontrollers to care about memory discipline.

If you're building AI features that need to run **everywhere**, these three OpenClaw variants are worth looking at.

And if you're not in that world yet? You will be. Product always wants one more surface area.

Bonus Articles

Local LLMs That Can Replace Claude Code

Small team of engineers can easily burn >$2K/mo on Anthropic's Claude Code (Sonnet/Opus 4.5). As budgets are tight, you...

medium.com

Observational Memory for Long-Running Agents — "This wasn't the plan!"

Always-on agents have unbounded context growth problem.

medium.com

Fully Autonomous Companies: OpenClaw Gateway + Routing + Agents

Fully Autonomous Companies: OpenClaw Gateway + Routing + Agents Whether you think it's hype or not, people are already...

medium.com

Deep Research with 96,000+ Trajectories — Completely Offline

Imagine synthesizing human-like research trajectories exceeding 100 turns entirely offline, no reliance on search or...

medium.com

ClawRouter: Anthropic charged me $4,660 — How I cut it 70% with smart LLM routing

Last month I opened my credit-card statement and almost threw up. Anthropic charged me $4,660.87, just for AI APIs.

medium.com

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison Just after 9:45 a.m. Pacific on 5 February 2026, Anthropic...

medium.com

Meta's AI-Scored Engineers: "Use AI or You're Out"

Meta's AI-Scored Engineers: "Use AI or You're Out" If you're shipping more than 50% of your code by hand without AI...

medium.com

Why Codex Became My Default Over Claude Code (for Now)

If you haven't tried Codex yet, I've got a brief take that might save you a few hours.

medium.com

Fully Local Agentic Coding on localhost: Claude Code, Codex, llama.cpp, Ollama and Docker MR

This is the setup I wish I had the first time I tried to run Claude Code and Codex-style workflows against local...

medium.com
#openclaw#picoclaw#zeroclaw#nanobots#ai-agent

Reporting a Problem

Sometimes we have problems displaying some Medium posts.

If you have a problem that some images aren't loading - try using VPN. Probably you have problem with access to Medium CDN (or fucking Cloudflare's bot detection algorithms are blocking you).

Problem Description Describe your problem here...

Submit Cancel